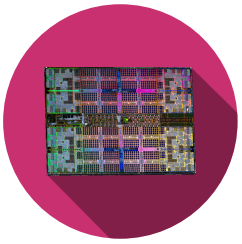


Single-Cycle RISC-V

Inhaltsverzeichnis

1 Einführung **1**

2 Architektur **2**
2.1 Analyse 2

3 Control Unit **2**
3.1 Umsetzung 3

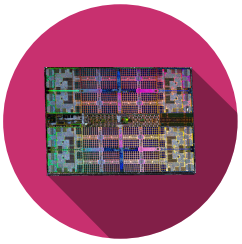
4 Simulation **3**
4.1 Analyse 3

5 Einsatz **4**

Akronyme **4**

1 Einführung

Dieses Labor nähert sich der RISC-V-Architektur mit Hilfe eines Prozessors, der einen reduzierten Befehlssatz unterstützt und in der Lage ist, ein kleines Assemblerprogramm auszuführen, indem er jeden Befehl in einem einzigen Clock ausführt (single-cycle). Dieser Prozessor namens HEIRV32_SC kann simuliert oder direkt auf einem FPGA-Chip eingesetzt werden. Die Verbindung zur Außenwelt kann über Tasten und Leds hergestellt werden.



2 Architektur

Abgesehen von der Inkrementierung des Programmzählers funktioniert der Rest rein kombinatorisch. Die Schaltung ist wie folgt:

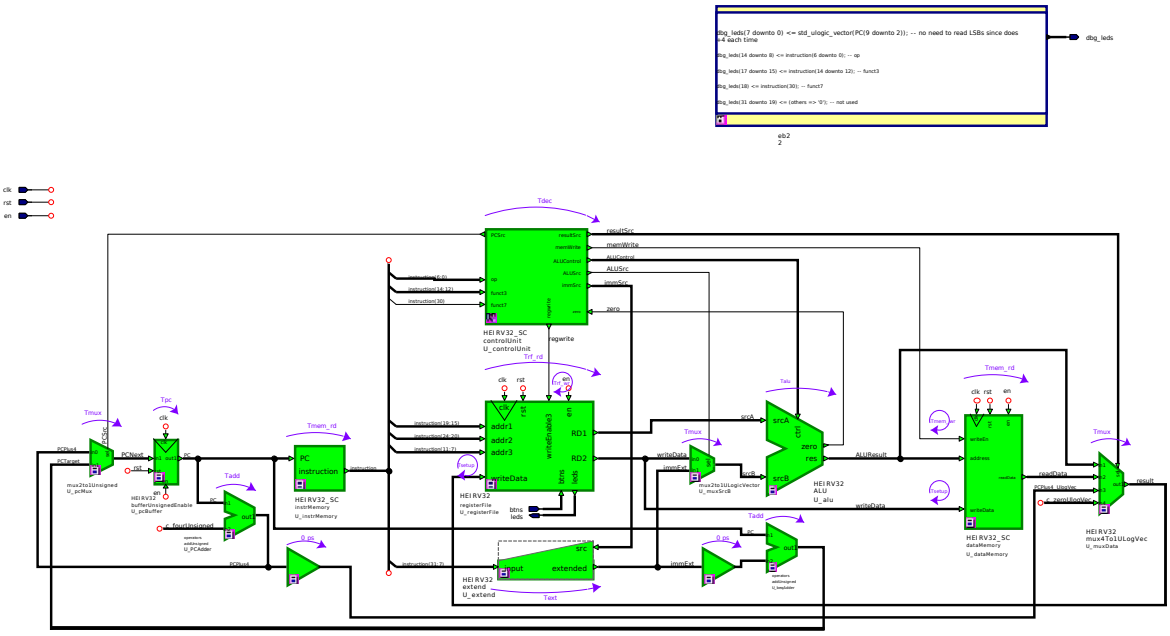


Abbildung 1: Top level

2.1 Analyse

Bestimmen Sie die verschiedenen Funktionen des Kreislaufs:

- Welche Rolle spielt jeder Block?
- Identifizieren Sie die fünf Schritte der RISC-Pipeline und welche Blöcke damit verbunden sind.
- Wozu dienen die einzelnen Signale? Wie groß sind sie?
- Wie werden die Signale des Blocks **controlUnit** für die Befehle **or, addi, sw, beq, jal** konfiguriert?

3 Control Unit

Eines der Hauptmerkmale, das einen Prozessortyp von einem anderen unterscheidet, ist die Art und Weise, wie die Befehle kodiert werden, was mit der internen Struktur zusammenhängt. Zum Beispiel gibt es die Operation, eine Zahl mit einem Register zu addieren, in der RISC-V-Architektur genauso wie in **x86** oder **ARM**, aber der erzeugte Opcode ist ganz anders. Für die Operation **addi x0 x0 1** in RISC-V (**add r0, r0, 1** in ARM) ist das Ergebnis :

- ARM : 0x 01 00 80 E2

- RISC-V : 0x 00 10 00 13

Der Block **controlUnit** ist derjenige, der dafür zuständig ist, die Schaltung gemäß der gegebenen Anweisung zu konfigurieren.

3.1 Umsetzung

Ergänzen Sie den Block **controlUnit**, um die folgenden Anweisungen zu unterstützen:

- R-type : add, sub, slt, or, and
- I-type : addi, slti, ori, andi, lw
- S-type : sw
- B-type : beq
- J-type : jal

4 Simulation

Unter dem Ordner **Simulation** finden Sie die Codes, die zur Simulation und zum Einsatz des Systems verwendet werden, in drei Formaten:

- .s: Assemblercode, unverschlüsselt
- .bin: kompilierter Code; kann mit Tools wie Ghidra oder Online-Disassemblern analysiert werden.
- .txt: kompilierte Version, die mit dem FPGA-Speicher für den Einsatz kompatibel ist.

Der Code wird mithilfe des Programms **HEIRV32-ASM** kompiliert, das zusammen mit der Dokumentation im gleichnamigen Ordner verfügbar ist.

Der Tester **heirv32_sc_tb** verwendet bei der Simulation den Code **code_sim.bin**.

4.1 Analyse

Analysieren Sie zunächst den Code **code_sim.s**, um seinen Inhalt zu verstehen. Ist es ein gutes Kandidat, um Ihren System zu testen? Nehmen Sie gegebenenfalls die notwendigen Änderungen vor. Bestimmen Sie die Zeit, die benötigt wird, um die Schleife des Programms einmal durchlaufen zu lassen, wenn man bedenkt, dass das System mit **66 MHz** getaktet ist.

Starten Sie die Simulation und bestätigen Sie, dass das System funktioniert.

Was sind die Schwächen dieser Single-Cycle-Architektur? Wie hoch ist die maximal denkbare Clockrate?

> Die Zeiten für die einzelnen Unterteile des Systems sind im Top-Level angegeben. Bestimmen Sie die längste Anweisung und verfolgen Sie ihren Weg zurück.

5 Einsatz

Beim Einsatz auf dem FPGA wählt das System den Code **code_sc.*** als Quelle aus. Analysieren und verstehen Sie den betreffenden Code. Was sollte passieren?

Die Bibliothek **Board** enthält den für den Einsatz benötigten Block. Öffnen Sie ihn und analysieren Sie die Schaltung: Wie funktionieren **en_step_n** und **en_full_n** ?

Die Datei **Board/concat/ebs2_heirv32.ucf** enthält die Definition und die Position der Pins des Systems. Der Anschluss der Elektronik erfolgt gemäß dem folgenden Bild:

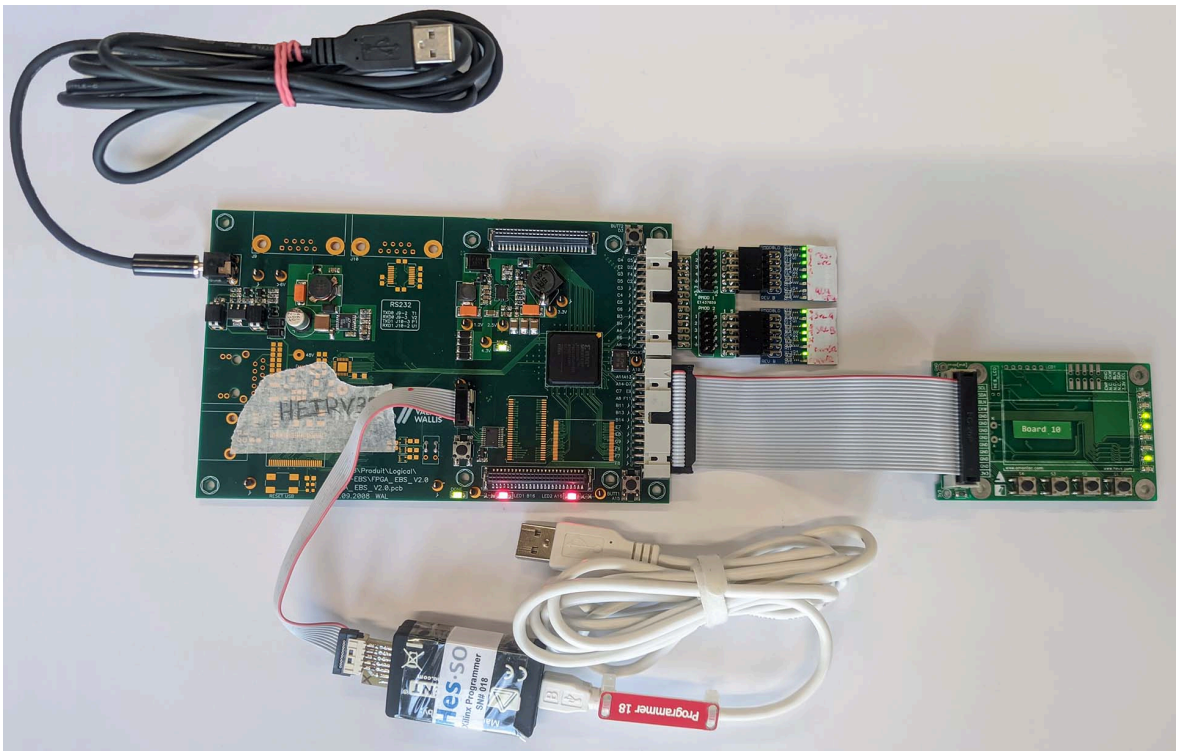


Abbildung 2: Anschließen der Hardware

Stellen Sie das System auf dem FPGA bereit und testen Sie es.

Beim Systemstart ist der Eintrag **en** von HEIRV32_SC auf 0 gesetzt!

Akronyme

FPGA **F**ield **P**rogrammable **G**ates **A**rray. 1, 3, 4

RISC-V **R**educed **i**nstruction **s**et **c**omputer **V**. 1, 2